

Nested Profile-Likelihood Inference of (A_s, A_c) from the Full Pixel Image

python-scripts/joint_profile_inference.py

Contents

1	Purpose and Relationship to Other Estimators	2
2	Per-Shot Pixel Likelihood (Recap)	2
2.1	Parameters	2
2.2	Pixel Rate and Poisson Likelihood	2
3	Pixel ACS via Exact CDF $\times$ 2D Gauss–Hermite	3
4	PSMAP Lookup: Catmull–Rom Interpolation and its Analytic Derivative	4
4.1	Value: 4D Cubic Interpolation	4
4.2	Analytic Gradient	4
4.3	Chaining Through to θ	5
5	Nested Profiling: Arrow Sparsity	5
6	Inner Solve: Per-Shot $(\phi_i, \theta_{i,0}, \theta_{i,100})$	6
6.1	Bounds (analytic path only)	6
6.2	Objective Rescaling	6
6.3	Convergence Check and the Envelope-Theorem Shortcut	6
7	Outer Solve: $\beta = (A_s, A_c)$	7
7.1	fit_beta — Per-Shot Python Loop	7
7.2	fit_beta_batch — Batched/GPU Inner Solve	7
8	Batching (pixel_acs_grad_batch.py)	7
9	Assumptions and Simplifications	8
10	Known Open Issues / Current Investigation	9
11	Reproducibility	9
11.1	CLI (fit_beta only)	9
11.2	Default Hyperparameters	10
11.3	Output	10

1 Purpose and Relationship to Other Estimators

This note documents `joint_profile_inference.py`, an estimator of the gradiometer signal $\beta = (A_s, A_c)$ (notation as in `notes/image_likelihood.tex`) that profiles *every* per-shot nuisance parameter — the common laser phase ϕ_i and both clouds’ 8-dimensional state $\eta_{i,z=0}, \eta_{i,z=100}$ — directly against the full pixel-binned Poisson likelihood, jointly with the outer optimisation over β , rather than against a reduced summary (port counts or image moments).

Why this exists. `crb_signal.py`’s Cramér–Rao analysis showed that the port-summed/moment-based pipeline (`map_inference.py`) retains only $\sim 9\%$ of the Fisher information available in ϕ_i once realistic cloud-parameter uncertainty is propagated, whereas the full pixel-resolved likelihood retains $\sim 96\%$ (each AI’s (ϕ, η) Fisher block goes from rank 1, in the port-summed reduction, to full rank 9 once individual pixel counts are used — see `crb_signal.pixel_fisher_block`). This script is the estimator designed to realise that gain in an actual per-shot fit, not just in a Fisher-information forecast.

Relationship to other pipelines in this repo.

- `map_inference.py` profiles a 4-moment Kalman summary of η_{iz} and marginalises ϕ_i on a grid (Section 2 of `notes/image_likelihood.tex`). Cheap, but discards spatial information beyond the first two moments.
- `full_image_is_inference.py` uses the full pixel image but marginalises η by importance sampling from a moment-posterior proposal (Section 7 of `notes/image_likelihood.tex`).
- **This script** uses the full pixel image and *profiles* (point-estimates, does not marginalise) η_{iz} and ϕ_i jointly, re-solving both at every trial β . It is the most computationally direct realisation of the full-information estimate, at the cost of the heaviest per-evaluation optimisation.

2 Per-Shot Pixel Likelihood (Recap)

This section restates, without re-deriving, the model shared with `notes/image_likelihood.tex` (Sections 1–3 there), fixing notation for what follows.

2.1 Parameters

- $\beta = (A_s, A_c)$: the two signal amplitudes (science parameters).
- ϕ_i : common laser phase for shot i , shared by both AIs, a priori $\text{Uniform}(0, 2\pi)$.
- $\delta\phi_i(\beta) = A_s \sin(2\pi f t_i) + A_c \cos(2\pi f t_i)$: the injected differential phase at AI $z = 100$ (zero at $z = 0$).
- $\theta_{iz} = (\mu_{x0}, \mu_{y0}, \mu_{vx}, \mu_{vy}, \sigma_{x0}, \sigma_{y0}, \sigma_{vx}, \sigma_{vy})_{iz} \in \mathbb{R}^8$: the initial phase-space Gaussian cloud parameters for AI $z \in \{0, 100\}$ in shot i (called **eta** elsewhere in the repo; renamed θ here to match the source file’s variable names).

The full per-shot parameter vector profiled by this script is

$$x_i = (\phi_i, \theta_{i,0}, \theta_{i,100}) \in \mathbb{R}^{17}. \quad (1)$$

2.2 Pixel Rate and Poisson Likelihood

For AI z , state $s \in \{g, e\}$ (ground/bright, excited/dark port group), pixel b :

$$\lambda_{izsb} = L_{iz} [A_{zsb}(\theta_{iz}) + C_{zsb}^c(\theta_{iz}) \cos \Phi_{iz} + C_{zsb}^s(\theta_{iz}) \sin \Phi_{iz}], \quad \Phi_{iz} = \phi_i + \delta\phi_i(\beta) [z=100], \quad (2)$$

with $n_{izsb} \sim \text{Poisson}(\lambda_{izsb})$ and L_{iz} the atom-number scale factor (plug-in normalisation matching predicted to observed total counts, Eq. 4 below; *not* a free parameter). The code’s variable names for (A, C^c, C^s) per state are `A_g, Cc_g, Cs_g` (state g) and `A_e, Cc_e, Cs_e` (state e), returned as `(n_bins,)` arrays by `acs_obj.pixel_acs(theta)` (Section 3).

The per-shot, per-AI negative log-likelihood implemented in `_ai_nll_only/_ai_nll_and_grad_analytic` is

$$-\log \mathcal{L}_{iz}(\theta_{iz}, \Phi_{iz}) = \sum_b \left[\mu_{gb} - n_{gb} \log \mu_{gb} \right] + \sum_b \left[\mu_{eb} - n_{eb} \log \mu_{eb} \right], \quad \mu_{sb} \equiv \lambda_{izsb}, \quad (3)$$

(constants $\log n!$ dropped), with the scale factor computed *inline, at every evaluation*, not cached from a separate profiling pass:

$$L_{iz} = \frac{\sum_b (n_{gb} + n_{eb})}{\max(\sum_b (I_{gb} + I_{eb}), \varepsilon)}, \quad I_{sb} = A_{sb} + C_{sb}^c \cos \Phi_{iz} + C_{sb}^s \sin \Phi_{iz}, \quad (4)$$

$\varepsilon = 10^{-300}$ (`crb.EPS`). Because L_{iz} is recomputed from (A, C^c, C^s) at the *current* (θ, Φ) every call, it is implicitly profiled out analytically at every point visited by the optimiser, not held fixed at a reference point as in the moment pipeline’s $\hat{\eta}$ -then-fix convention.

The joint per-shot objective actually minimised (`joint_nll/joint_nll_and_grad_analytic`) is

$$F_i(x_i; \delta\phi_i^{\text{trial}}) = -\log \mathcal{L}_{i,0}(\theta_{i,0}, \phi_i) - \log \mathcal{L}_{i,100}(\theta_{i,100}, \phi_i + \delta\phi_i^{\text{trial}}) + \frac{1}{2} \left\| \frac{\theta_{i,0} - \bar{\theta}}{\sigma_\theta} \right\|^2 + \frac{1}{2} \left\| \frac{\theta_{i,100} - \bar{\theta}}{\sigma_\theta} \right\|^2, \quad (5)$$

where $\bar{\theta}, \sigma_\theta$ are the (identical, independent) Gaussian prior mean/std for both AIs’ clouds (`prior_mean, prior_std` arguments; matches the dataset’s generation hyperparameters, van Trees / Bayesian-profiling convention, same as Section 2.3 of `notes/image_likelihood.tex`). ϕ_i carries *no* prior term (flat), consistent with $\phi_i \sim \text{Uniform}(0, 2\pi)$ — but note that here ϕ_i is *profiled* (point-estimated) jointly with θ , not *marginalised* over a phase grid as in `map_inference.py/full_image.is`. This is a genuine modelling difference, not just an implementation shortcut: see §9.

3 Pixel ACS via Exact CDF $\times$ 2D Gauss–Hermite

$(A_{zsb}, C_{zsb}^c, C_{zsb}^s)$ are computed by `SemiAnalyticPixelACS.pixel_acs(profile_cloud_nuisances.py)`, identical to the evaluator already documented in Section 7.3 of `notes/image_likelihood.tex`; restated compactly here since it is the main cost driver.

1. **Detection-plane Gaussian.** Free ballistic flight for time $T = t_{\text{det}}$:

$$\mu_{xf} = \mu_{x_0} + T\mu_{vx}, \quad \sigma_{xf}^2 = \sigma_{x_0}^2 + (T\sigma_{vx})^2, \quad (6)$$

and identically for y .

2. **Exact spatial weight.** Bin occupancy probability via the Gaussian CDF (no quadrature error, never zero):

$$P_b = \left[\Phi\left(\frac{x_{b,\text{hi}} - \mu_{xf}}{\sigma_{xf}}\right) - \Phi\left(\frac{x_{b,\text{lo}} - \mu_{xf}}{\sigma_{xf}}\right) \right] \times [y\text{-analogue}]. \quad (7)$$

3. **Conditional velocity distribution at the bin centre** (x_b, y_b) (standard Gaussian conditioning, decoupled in x/y):

$$v_{x_0} \mid x_b \sim \mathcal{N}\left(\mu_{vx} + \frac{T\sigma_{vx}^2}{\sigma_{xf}^2}(x_b - \mu_{xf}), \frac{\sigma_{vx}^2 \sigma_{x_0}^2}{\sigma_{xf}^2}\right), \quad (8)$$

and identically for $v_{y_0} \mid y_b$.

4. **2D Gauss–Hermite quadrature** (n_{gh} nodes per velocity dimension, $n_v = n_{\text{gh}}^2$ nodes total per bin) over this conditional distribution gives the initial phase-space sample points $(x_0, y_0, v_{x_0}, v_{y_0})$ fed to the PSMAP lookup (§4), one full grid of n_v nodes *per bin*, processed `chunk_bins` bins at a time to bound GPU memory.

5. **Port sum and GH average.** PSMAP outputs $(\Delta\phi_p, a_{0p}, a_{1p})$ per port p (§4) give

$$A_p = a_{0p}^2 + a_{1p}^2, \quad C_p^c = \iota_p 2a_{0p}a_{1p} \cos \Delta\phi_p, \quad C_p^s = -\iota_p 2a_{0p}a_{1p} \sin \Delta\phi_p, \quad (9)$$

($\iota_p = \text{port_inter}$, an interference sign/weight per port), summed over the ports belonging to state s (boolean masks `s0_g`, `s1_g`), GH-averaged over the n_v velocity nodes with weights w_2 , and multiplied by P_b :

$$A_{zsb} = P_b \sum_{v=1}^{n_v} w_{2v} \sum_{p \in s} A_p(\text{node } v), \quad (\text{and likewise for } C_{zsb}^c, C_{zsb}^s). \quad (10)$$

Default $n_{\text{gh}} = 20$ in `profile_cloud_nuisances.py`’s own CLI, but `joint_profile_inference.py` passes it through as `pixel_n_gh` with a much smaller default of **4** ($n_v = 16$ nodes/bin) — see §11 for the cost/accuracy trade-off this implies.

4 PSMAP Lookup: Catmull–Rom Interpolation and its Analytic Derivative

This section is *not* covered elsewhere in the notes tree and is specific to the analytic-gradient machinery (`pixel_acs_grad.py`, `pixel_acs_grad_batch.py`) built for this estimator.

4.1 Value: 4D Cubic Interpolation

The precomputed PSMAP is a 4D grid over (x, y, v_x, v_y) , one grid per "port" $p = 1, \dots, n_P$ (a port being one interfering/non-interfering branch of the atom-interferometer sequence), storing the accumulated phase $\Delta\phi_p$ and two beam-splitter amplitudes a_{0p}, a_{1p} . For a query point $(x_0, y_0, v_{x_0}, v_{y_0})$, each of the 4 coordinates is located in its grid cell with a fractional offset $t \in [0, 1]$, and the interpolated value is a **Catmull–Rom cubic** tensor product over the $4^4 = 256$ surrounding corners:

$$\hat{f}(x_0, y_0, v_{x_0}, v_{y_0}) = \sum_{k_x, k_y, k_{v_x}, k_{v_y}=0}^3 w_x(t_x)_{k_x} w_y(t_y)_{k_y} w_{v_x}(t_{v_x})_{k_{v_x}} w_{v_y}(t_{v_y})_{k_{v_y}} f[\text{corner}], \quad (11)$$

where each $w(\cdot)_k$, $k = 0, \dots, 3$ (stencil offsets $-1, 0, 1, 2$ from the cell floor), is the standard Catmull–Rom cubic-polynomial weight in t . This is the same interpolation scheme as `SurrogatePixelACS._eval_fast` documented for PSMAPSurrogate generally; what is new here is the gradient.

4.2 Analytic Gradient

Because each $w(t)_k$ is an explicit cubic polynomial in t , its derivative $w'(t)_k$ is also a closed-form polynomial (`_cr_weights_deriv`), so

$$\frac{\partial \hat{f}}{\partial x_0} = \frac{1}{\Delta x} \sum_{\text{corners}} w'_x(t_x)_{k_x} w_y(t_y)_{k_y} w_{v_x}(t_{v_x})_{k_{v_x}} w_{v_y}(t_{v_y})_{k_{v_y}} f[\text{corner}], \quad (12)$$

and symmetrically for y_0, v_{x_0}, v_{y_0} — an *exact* derivative of the interpolant, not a finite-difference approximation, at a cost of one extra weighted gather per coordinate (4 total) rather than re-touching the expensive per-port stacked-grid gather with perturbed inputs.

Two implementation details, both load-bearing for correctness:

- **Boundary clipping.** `find_cell` clips both the cell index and the fractional coordinate t at the grid edge, so the interpolated *value* is pinned flat outside the domain (extrapolation-by-constant). The raw polynomial derivative does not know about this and would return a spurious nonzero slope there; it is explicitly zeroed for out-of-bounds points (`in_x`, `in_y`, `in_vx`, `in_vy` masks) to match the true (flat) value function.
- **Catastrophic cancellation in $\Delta\phi$.** The phase field $\Delta\phi_p$ is an unwrapped accumulated phase, $O(10^8)$ in magnitude, unlike $a_0, a_1 = O(1)$. The 256 corner weights used for a *derivative* sum to exactly zero in exact arithmetic, so summing $w'_k \Delta\phi[\text{corner}_k]$ directly loses essentially all significant digits to cancellation. Subtracting a common per-query reference value from all 256 corners before the weighted sum (`rebase=True` in `_gather_sum`) is mathematically identical (the weights still sum to zero, so a constant shift is annihilated) but removes the cancellation. This rebasing is applied *only* to the four derivative gathers, not the value gather.

4.3 Chaining Through to θ

The map $\theta \rightarrow (x_0, y_0, v_{x_0}, v_{y_0})$ (cloud parameters to PSMAP sample points, §3, steps 1–3) is pure elementary arithmetic — it never touches the PSMAP lookup — so its Jacobian is obtained by *cheap* central finite differences (step h_θ , hardcoded 10^{-9} in the batched path, 10^{-7} in an unused/dead local variable in `fit_beta` — see §9) and chained with the analytic $\partial\hat{f}/\partial(x_0, y_0, v_{x_0}, v_{y_0})$ from Eq. 12 via the chain rule. This hybrid (analytic-through-the-expensive-part, finite-difference-through-the-cheap-part) is what `pixel_acs_and_grad/pixel_acs_and_grad_batched` compute, returning both the 6-tuple $(A_g, C_g^c, C_g^s, A_e, C_e^c, C_e^s)$ and its $(8, 6, n_{\text{bins}})$ (single-shot) or $(N, 8, 6, n_{\text{bins}})$ (batched) gradient w.r.t. θ . $\partial/\partial\phi$ is fully analytic (plain cos/sin chain rule on Eq. 2), since ϕ enters the model only trigonometrically and never touches the PSMAP call.

5 Nested Profiling: Arrow Sparsity

Collect all shots’ nuisances and the signal into one vector $(\beta, x_1, \dots, x_{N_{\text{shots}}}) \in \mathbb{R}^{2+17N}$. Because shot i ’s likelihood depends on x_i and β only (never on $x_j, j \neq i$), the joint Fisher/Hessian matrix has exact **arrow sparsity**: the $(2 + 17N) \times (2 + 17N)$ matrix is dense only in the leading 2×2 block and the first two rows/columns; every x_i block is block-diagonal and couples *only* to β , never to another shot.

This licenses an exact reduction (not an additional approximation) of the joint MLE to:

outer step on β (2 parameters), each evaluation requiring an **inner** per-shot solve of $x_i = (\phi_i, \theta_{i,0}, \theta_{i,100})$ (17 parameters, N_{shots} independent, embarrassingly parallel problems), warm-started from the previous outer iteration’s solution.

This is explicitly *not* the same as profiling θ once at a reference β and holding it fixed (valid for the moment pipeline, Section 2.4 of `notes/image_likelihood.tex`, because η is only weakly coupled to β there): the module docstring records that θ (especially the weakly-identified ridge/width directions, Section 4.2 of `notes/image_likelihood.tex`) was empirically observed to shift by 6–24 posterior- σ as β sweeps a realistic range, so θ *must* be re-profiled at (or near) every trial β visited by the outer optimiser — hence the nested structure rather than a two-pass profile-then-fix scheme.

6 Inner Solve: Per-Shot ($\phi_i, \theta_{i,0}, \theta_{i,100}$)

Given a trial $\delta\phi_i^{\text{trial}} = \delta\phi_i(\beta)$ and a warm-start $x_i^{(0)}$ (from the previous outer iteration; initialised at $(\phi = 0, \theta_{i,0} = \bar{\theta}, \theta_{i,100} = \bar{\theta})$ on the very first outer evaluation), the inner solve is

$$\hat{x}_i(\beta) = \arg \min_{x_i} F_i(x_i; \delta\phi_i^{\text{trial}}) \quad (13)$$

via bounded L-BFGS-B (`scipy.optimize.minimize`). Two implementations exist, differing only in how the gradient is supplied:

- `profile_shot (joint_nll)`: `scipy`'s own internal finite-difference gradient. Slower, but with "much less risk of a subtle sign/indexing bug" (module docstring) — the deliberately simple reference path. **No bounds are passed** to this `minimize` call (an asymmetry with the analytic path below; see §9).
- `profile_shot_analytic (joint_nll_and_grad_analytic)`: the hybrid analytic gradient of §4, supplied via `jac=True`.

6.1 Bounds (analytic path only)

$$\phi_i \in [-50, 50] \text{ rad (effectively unconstrained)}, \quad \theta_{iz,k} \in \bar{\theta}_k \pm 20 \sigma_{\theta,k} \quad \forall k. \quad (14)$$

These are numerical safety rails, not part of the statistical model: an unbounded solve was observed directly to diverge (gradient norm *growing* with more iterations rather than shrinking) by chasing a spurious unbounded-improvement direction, motivating the generous but finite θ bound.

6.2 Objective Rescaling

Per-shot NLL is $O(10^7)$ (high photon counts), so raw gradients are $O(10^5\text{--}10^8)$, well outside the regime L-BFGS-B's default step-size heuristics (tuned for $O(1)$ objectives) are designed for. The objective and gradient are both rescaled by a constant $c = 1/\max(\|g_{\text{probe}}\|, 1)$, computed once (cached in `_scale_cache`) from the gradient at the very first call, before the optimiser sees them. This is a pure rescaling — it does not move the argmin — but is essential for L-BFGS-B's step-size selection to behave sanely.

6.3 Convergence Check and the Envelope-Theorem Shortcut

After `n_iter` L-BFGS-B iterations, the *unscaled* gradient at the reported solution is recomputed and its norm compared against `grad_tol` $\times \max(|\text{NLL}|, 1)$; if this is exceeded, a `UserWarning` is raised ("*inner solve NOT converged ... envelope-theorem outer gradient is unreliable here*").

This check exists because `profile_shot_analytic` also returns

$$\left. \frac{\partial \text{NLL}_{i,100}}{\partial \delta\phi_i^{\text{trial}}} \right|_{\hat{x}_i}, \quad (15)$$

which, **by the envelope theorem**, equals $\partial F_i / \partial \delta\phi_i^{\text{trial}}$ exactly *at a true stationary point* of the inner objective (the $z=0$ contribution and all implicit $\partial \hat{x}_i / \partial \delta\phi_i^{\text{trial}}$ terms vanish there). This is what lets the outer gradient be computed *without* differentiating through the inner `arg min` — but the identity is only exact at convergence, so the diagnostic in this subsection is not optional decoration: an unconverged inner solve silently produces a biased outer gradient with no other symptom.

7 Outer Solve: $\beta = (A_s, A_c)$

Bounded L-BFGS-B over $\beta \in [-\text{grid_half}, \text{grid_half}]^2$ (default `grid_half` = 0.2 rad), multi-start from $\{(0, 0)\} \cup \{(0.05, 0), (-0.05, 0.05)\}[:\text{n_starts}-1]$, best of the `n_starts` local solutions kept. **Two outer implementations exist**, and they are *not* structurally symmetric:

7.1 `fit_beta` — Per-Shot Python Loop

For each trial β : loop over shots, calling `profile_shot` or `profile_shot_analytic` once per shot (warm-started from `state['x'][k]`, updated in place), summing the per-shot NLL.

- **`use_analytic=False`**: `neg_logL` alone, no `jac=True` — scipy finite-differences the *outer* objective too (on top of the FD inner solves via `profile_shot`).
- **`use_analytic=True`** (default): `neg_logL` and `grad_analytic`, which additionally accumulates the outer gradient via the envelope theorem (§6.3):

$$\frac{\partial(-\log \mathcal{L})}{\partial A_s} = \sum_i \frac{\partial \text{NLL}_{i,100}}{\partial \delta \phi_i} \sin(2\pi f t_i), \quad \frac{\partial(-\log \mathcal{L})}{\partial A_c} = \sum_i \frac{\partial \text{NLL}_{i,100}}{\partial \delta \phi_i} \cos(2\pi f t_i), \quad (16)$$

supplied via `jac=True`. The outer objective/gradient are additionally rescaled by a constant `obj_scale`, chosen once (from the raw gradient norm at $\beta = 0$) so that L-BFGS-B's first step is $O(0.02)$ in β -space rather than overshooting by orders of magnitude (same rationale as the inner rescaling, applied at the outer level too).

7.2 `fit_beta_batch` — Batched/GPU Inner Solve

Instead of a Python loop, all N shots' x_i are stacked into one $(17N,)$ vector and solved with a *single* L-BFGS-B call per outer β evaluation (`batched_nll_grad`, using `pixel_acs_grad_batch.pixel_acs_and_grad` §8), amortising GPU kernel-launch overhead across shots.

Critically, `fit_beta_batch`'s outer `minimize` call

```
minimize(neg_logL, b0, method='L-BFGS-B', bounds=bounds, options={'maxiter':
                                outer_maxiter})
```

had **no `jac=True` and no objective rescaling**: unlike `fit_beta`, there was no envelope-theorem outer gradient and no `obj_scale` analogue, so scipy finite-differenced the *outer* objective itself, with each FD probe triggering a fresh, warm-started, only-`n_inner_iter`-step inner batched solve. **This has since been fixed** (§10): `batched_nll_grad` now also returns the per-shot $\partial \text{NLL}_{100} / \partial \phi$ needed for the envelope theorem, and `fit_beta_batch` mirrors `fit_beta`'s analytic outer gradient, objective rescaling, and theta bounds. A second, structural problem remains unfixed — see §10.

8 Batching (`pixel_acs_grad_batch.py`)

"Batching" here means stacking the leading axis of every per-shot quantity so that the four `_eval_fast_grad` calls (one per grid dimension's derivative gather, §4) and the finite-difference θ -Jacobian loop are each issued *once* across all N shots simultaneously, rather than once per shot in a Python loop.

Quantity	Single-shot (<code>pixel_acs_grad.py</code>)	Batched (<code>pixel_acs_grad_batch.py</code>)
θ	$(8,)$	$(N, 8)$
Sample points $x_0, y_0, v_{x_0}, v_{y_0}$	$(n_{\text{bins}} n_v,)$	$(N, n_{\text{bins}} n_v)$
PSMAP lookup input (raveled)	$(n_{\text{bins}} n_v,)$	$(N n_{\text{bins}} n_v,)$ — one call
Base ACS $(A_g, \dots, C_e^s)$	$6 \times (n_{\text{bins}},)$	$6 \times (N, n_{\text{bins}})$
θ -gradient	$(8, 6, n_{\text{bins}})$	$(N, 8, 6, n_{\text{bins}})$

The θ -Jacobian’s finite-difference loop (`for k in range(8)`) still perturbs *one component at a time*, but perturbs it for *all N shots simultaneously* in that iteration (`tp[:, k] += h_theta[k]`, a scalar step broadcast across the shot axis) — so it is still 8 sequential perturbation calls total, not $8N$, each itself a batched (all-shots) `_sample_points_batch` call. The finish/port-sum/GH-average tail (`_finish_batch`) is likewise a straight axis-broadcast of the single-shot `_finish` logic, with an explicit $(N, n_{\text{bins}}, n_v)$ reshape inserted before the GH weight contraction.

The module’s own docstring flags this file as *”the highest-risk part of the session’s speedup work, validated step by step against the already-correct per-shot loop before trusting it”* — no such validation script currently exists in the repo (as of this writing); see §10.

9 Assumptions and Simplifications

- **ϕ_i is profiled, not marginalised.** Unlike `map_inference.py/full_image_is_inference.py`, which integrate out ϕ_i over a phase grid (Section 2.3 of `notes/image_likelihood.tex`) precisely to avoid the per-shot phase absorbing the signal and biasing $\hat{\beta}$ toward zero, this estimator point-estimates ϕ_i jointly with θ . This is a genuine modelling choice (not just cost-driven), and its effect on $\hat{\beta}$ ’s bias/variance relative to the marginalised estimators has not yet been characterised in this file or its notes — flagged as an open question, not an established equivalence.
- **L_{iz} (flux normalisation) is analytically profiled at every evaluation**, not a free optimised parameter and not cached from a separate pass (§2).
- **Independent Gaussian priors** on $\theta_{i,0}$ and $\theta_{i,100}$, matching the dataset’s generation hyperparameters, with *no cross-AI covariance term*.
- **Per-shot inner objective assumed effectively unimodal.** There is no multi-start at the per-shot (inner) level, only at the outer β level; the warm-start scheme also implicitly assumes the optimum moves continuously (and slowly, relative to `n_inner_newton/n_inner_iter` steps) as β is perturbed by the optimiser or by finite differences.
- **Bound constraints are numerical rails, not modelling choices** (§6).
- **Known inconsistencies in the source, noted for reproducibility:**
 - `fit_beta`’s `gh_order` parameter (and the matching `--gh_order` CLI flag) is accepted but **never used** inside `fit_beta` — it is a leftover from the port-summed API in `crb_signal.py` and has no effect on this script’s behaviour.
 - `fit_beta` defines a local `h_theta = np.full(8, 1e-7)` that is **never passed anywhere** (dead code); the finite-difference step actually used for the analytic θ gradient is the module-level constant `H_THETA_ANALYTIC = np.full(8, 1e-9)` in `pixel_acs_grad.py`. `fit_beta_batch` does define and use its own $h_\theta = 10^{-9}$ correctly.
 - `profile_shot` (the plain finite-difference inner solve) passes **no bounds** to `scipy.optimize.minimize` (default: unconstrained), whereas `profile_shot_analytic` always bounds θ and ϕ

(§6). The two inner-solve implementations are therefore not drop-in equivalent even before considering gradient source.

10 Known Open Issues / Current Investigation

`fit_beta_batch` was reported to run faster than `fit_beta` but converge to a different $\hat{\beta}$. Investigation resolved this into **two distinct issues**.

Issue 1 — fixed. Direct numerical cross-validation of `pixel_acs_grad_batch.py` against the per-shot loop (`pixel_acs_grad.py`), for several random shots and θ values, showed the batched value *and* gradient are **byte-identical** (max abs. difference = 0.0) to the single-shot reference — the batched pixel-level math in §3–4 is **not** the source of the discrepancy, despite the module docstring’s own admission that this had never actually been checked. The real bug was exactly hypothesis (2) originally raised in this section: `fit_beta_batch`’s outer `minimize` call had no analytic gradient, so `scipy`’s own finite-difference outer gradient was computed by perturbing β and re-solving the warm-started (`state['x']` mutated in place) inner problem at each FD probe — making repeated evaluations near the same β history-dependent and corrupting the FD gradient estimate. This has been **fixed**: `batched_nll_grad` now additionally returns the per-shot $\partial \text{NLL}_{100} / \partial \phi$, `fit_beta_batch` now assembles the same envelope-theorem outer gradient as `fit_beta` (§7) and applies matching objective rescaling and θ/ϕ bounds on its inner solve.

Issue 2 — found, not fixed; structural. Even with Issue 1 fixed, forcing the inner batched solve to hard convergence (up to 1000 iterations) on real data revealed that the single monolithic ($17N$)-dimensional L-BFGS-B call converges markedly *worse* than N independent per-shot 9-dimensional solves: in a 4-shot test, one shot’s ϕ got stuck in the wrong local optimum ($\phi \approx 0.77$ vs. the loop’s $\phi \approx 0.00$) with a residual gradient norm ($\sim 8 \times 10^7$) that did not shrink with further iterations. This is not a convergence *tolerance* problem (more `n_inner_iter` does not fix it): it is that bundling all shots into one L-BFGS-B call shares curvature/step-size history across shots whose ϕ -landscapes are independently multi-modal, so one “hard” shot can stall the whole batch. This defeats the purpose of the arrow-sparsity argument in §5, which licenses solving the shots *independently* given β — the batched implementation currently does not exploit that independence at the optimizer level, only at the value/gradient-evaluation level. A correct fix (hand-vectorised per-shot quasi-Newton steps, sharing only the batched GPU value/gradient call but keeping independent per-shot optimiser state) is a larger rewrite, out of scope for the fix applied so far.

Bottom line. The Fisher-theoretic motivation in §1 (pixel likelihood retains $\sim 96\%$ vs. $\sim 9\%$ of the phase information) is a property of the *model*, independent of either bug above. `fit_beta` (the per-shot loop), given adequate inner/outer iteration budgets (§11), is validated and should be trusted as the realisation of that gain. `fit_beta_batch`, even after Issue 1’s fix, should **not yet** be treated as a faster drop-in replacement until Issue 2 is addressed — it can converge to the wrong local optimum for shots with a hard ϕ landscape, silently.

11 Reproducibility

11.1 CLI (`fit_beta` only)

The `__main__` block only exercises `fit_beta`; there is **no CLI flag to invoke `fit_beta_batch`** — it must be called directly from Python (as done by the ad hoc scripts used in the investigation of §10).

```
python python-scripts/joint_profile_inference.py \
  --data_root <path/to/dataset/root> \
  --n_shots 5 --bins 16 --pixel_n_gh 4 --f_signal 0.3 \
  --n_inner_newton 3 --n_starts 1 --outer_maxiter 15 --use_analytic 1
```

`data_root` must contain at least one `run_*` directory with `Z0/data_IMG.h5` and `Z100/data_IMG.h5` (the standard `ImageShotDataset` layout used throughout the repo); the script uses `sorted(data_root.glob('run_*'))` i.e. always the first run. The hardcoded prior in `_main_` is $\bar{\theta} = (0, 0, 0, 0, 100, 100, 100, 100) \mu\text{m}/(\text{s})$, $\sigma_{\theta} = 10 \mu\text{m}/(\text{s})$ for all 8 components — override by calling `fit_beta/fit_beta_batch` directly if the dataset’s true generation hyperparameters differ.

11.2 Default Hyperparameters

Parameter	CLI flag	Default
Shots used	<code>--n_shots</code>	5
Inference bins per side	<code>--bins</code>	16
GH order per velocity dim	<code>--pixel_n_gh</code>	4 ($n_v = 16$)
Signal frequency f	<code>--f_signal</code>	0.3
Inner solve iterations	<code>--n_inner_newton</code>	3
Outer multi-starts	<code>--n_starts</code>	1
Outer L-BFGS-B iterations	<code>--outer_maxiter</code>	15
Analytic gradient	<code>--use_analytic</code>	1 (True)
Outer bound (<code>grid_half</code>)	— (not exposed via CLI)	0.2 rad

Note the very small default inner-iteration budgets (`n_inner_newton=3`, or `n_inner_iter=15` for the batched path): given the $O(10^7)$ NLL scale and rescaling discussed in §6, these defaults may not be sufficient for the inner solve to actually satisfy the convergence check of §6.3 at every shot/outer-iteration, in which case the envelope-theorem outer gradient (§7) is not reliable regardless of which outer implementation is used — always check for the “*inner solve NOT converged*” warning when reproducing results from this script, and increase the inner iteration budget if it fires.

11.3 Output

Currently the script only prints $\hat{\beta}$ to stdout (`print(f'\nFinal beta_hat (As, Ac) = {beta_hat}...')`); there is no structured (JSONL/pickle) output file, unlike `map_inference.py` and `profile_cloud_nuisances.py`. Anyone running this at scale (multiple runs/shots for a distribution, as in `notebooks/map_mle_distributions.py` treatment of the other estimators) will need to add that themselves.